

BDP × Window

物理管道容量怎样生成窗口需求

408 · NET-B05

Mother Interface

Path rate/time → required in-flight data → window bound → utilization

BDP 是某段传播期间可注入的数据量；可靠窗口是在反馈到达前允许保持的未确认数据量。两者通过“反馈管道需要多少数据才能不空”连接。

Owners 与边界

NET01 Owns R, T_{prop}, RTT 与 delay decomposition; NET03 Owns window、ACK 与流水线。本桥 Owns 量纲转换和最小窗口条件。rwnd/cwnd 如何限制实际窗口交给 NET-B04; 队列拥塞交给 NET07。

1 先统一时间口径

若只问“单向链路上同时传播的数据”，则 $BDP = R T_{prop}$ 。若问发送方从开始发送一个 frame 到其 ACK 返回前的停等周期，就要包含题设中的发送、往返传播、ACK 发送及其他处理时间：

$$T_{cycle} = T_{tx,data} + 2T_{prop} + T_{tx,ack} + T_{other}.$$

窗口要让发送器在整个反馈周期内持续忙碌，所需数据量近似为 $R T_{cycle}$ ；如果题目把“窗口”只计未确认 data frames，还应按 frame 长度离散取整。

2 从停等到流水线

设 data frame 长 L bit，发送时间 $T_f = L/R$ ，忽略 ACK 发送与处理。停等利用率为

$$U_{sw} = \frac{T_f}{T_f + 2T_{prop}}.$$

窗口允许最多 W 个 frame 在途时，理想化发送端利用率为

$$U = \min \left\{ 1, \frac{W T_f}{T_f + 2T_{prop}} \right\}.$$

要达到满利用率：

$$W \geq \left\lceil \frac{T_f + 2T_{prop}}{T_f} \right\rceil = \left\lceil 1 + 2 \frac{T_{prop}}{T_f} \right\rceil.$$

式中的 1 对应首帧本身的发送时间；若题目另定义 RTT 从“首帧发送完”起算，公式外观会变，但事件时间线必须等价。

3 BDP 只是必要容量，不是实际吞吐保证

窗口达到反馈管道需求，只消除了“等 ACK 导致发送器空闲”这一瓶颈。实际吞吐还可能受 source rate、receiver rwnd、congestion cwnd、loss/retransmission、最慢链路和协议开销限制：

$$\text{Throughput} \leq \min \left(R_{bottleneck}, \frac{W_{effective}}{RTT} \right).$$

这是一条上界接口，不是所有题目都能直接取等。RTT 增大时，同一窗口能支撑的吞吐下降；链路速率增大但窗口不变时，也可能出现“带宽升级、吞吐不升”。

4 数值母例

链路 $R = 100 \text{ Mbps}$ ，单向传播 $T_{prop} = 20 \text{ ms}$ ，data frame 为 10,000 bit，忽略 ACK: $T_f = 0.1 \text{ ms}$ ，反馈周期约 40.1 ms，需要

$$W_{min} = \left\lceil \frac{40.1}{0.1} \right\rceil = 401 \text{ frames.}$$

对应约 4.01 Mbit 未确认数据。只计算单向 $RT_{prop} = 2 \text{ Mbit}$ 会低估可靠发送方等待 ACK 的在途需求，因为反馈要走完往返路径。

5 调用协议与 First Divergence

1. 画“开始发首帧—发完—到达—ACK 返回”时间线；
2. 标明 RTT/传播时延的题设定义，列入 ACK 与处理开销；
3. 把窗口统一成 bit、byte 或 frame，做量纲检查并向上取整；
4. 算满利用率所需窗口，再与题设实际有效窗口比较；
5. 若未达理论吞吐，继续检查 NET-B04 与 bottleneck，而非强行取等。

Anti-Bridge

BDP 当成一种时延，是量纲错误；用单向 BDP 直接回答 ACK 窗口，是漏了反馈返回；窗口大于 BDP 就断言不会拥塞，是把物理填管需求变成网络容量许可；忘记 frame 离散取整，会少一个窗口槽。

Compression

窗口覆盖的是单向传播还是完整反馈周期？速率与时间的口径是什么？需要多少在途 bit/frame？实际窗口又被谁进一步收紧？